

โครงการปลายภาค: การพัฒนาระบบ Data Engineering แบบ End-to-End

ภาพรวมโครงการ

โครงการนี้มีวัตถุประสงค์ให้นักศึกษาออกแบบและพัฒนา ระบบ Data Engineering แบบ End-to-End โดยนำข้อมูลจากหลายแหล่งมาผ่านกระบวนการจัดเก็บ แปลง และโหลดเข้าสู่ระบบคลังข้อมูล (Data Warehouse) เพื่อให้ได้ข้อมูลที่พร้อมสำหรับการนำไปใช้งานเชิงวิเคราะห์

โครงการมุ่งเน้นทักษะด้าน:

- การนำเข้าข้อมูล (Data Ingestion)
- การพัฒนา ETL / ELT Pipeline
- การออกแบบ Data Warehouse
- การประมวลผลแบบเพิ่มข้อมูลใหม่ (Incremental Processing)
- การตรวจสอบคุณภาพข้อมูล (Data Quality)
- การทำงานแบบอัตโนมัติ (Orchestration)

เน้น Data Engineering เป็นหลัก ไม่เน้น Machine Learning หรือการวิเคราะห์ขั้นสูง

วัตถุประสงค์การเรียนรู้

เมื่อสิ้นสุดการทำโครงการ นักศึกษาจะสามารถ:

- ออกแบบสถาปัตยกรรมระบบข้อมูลแบบ End-to-End โดยระบุแหล่งข้อมูล กระบวนการประมวลผล และโครงสร้างการจัดเก็บข้อมูลได้อย่างชัดเจน
- พัฒนา ETL/ELT Pipeline แบบหลายขั้นตอน สำหรับการดึง แปลง และโหลดข้อมูลเข้าสู่ระบบ
- รวมและจัดการข้อมูลจากหลายแหล่ง (เช่น File, API, Database) ให้สามารถใช้งานร่วมกันได้อย่างถูกต้องและสอดคล้องกัน
- ออกแบบโครงสร้าง Data Warehouse ด้วยหลัก Dimensional Modeling (Star หรือ Snowflake Schema) เพื่อรองรับการวิเคราะห์เชิงธุรกิจ
- พัฒนาระบบการโหลดข้อมูลแบบ Incremental โดยสามารถเพิ่ม อัปเดต และป้องกันข้อมูลซ้ำได้
- กำหนดและตรวจสอบกฎคุณภาพข้อมูล (Data Quality Rules) เช่น ความครบถ้วน ความถูกต้อง และความไม่ซ้ำของข้อมูล
- ออกแบบให้ระบบ Pipeline สามารถทำงานอัตโนมัติ พร้อมมีการจัดการ Logging และการจัดการข้อผิดพลาดเบื้องต้น

การทำงานเป็นทีม

- ทีมละ 2 – 3 คน
- ลงชื่อสมาชิกในทีมที่ Link -> [682-ITDS344-Data Engineering Project Registration.xlsx](#)

แนวทางการดำเนินโครงการ (Project Instructions)

Phase 1: Business Understanding & Data Planning (สัปดาห์ที่ 1 - 2)

เป้าหมาย กำหนดโจทย์ธุรกิจ และออกแบบสถาปัตยกรรมระบบแบบครบวงจร (End-to-End)

1. ศึกษาและวิเคราะห์ระบบเดิม/ระบบที่เกี่ยวข้อง (Existing System & Market Research)

- สืบค้นและศึกษาว่าปัจจุบันมีระบบที่แก้ปัญหาลักษณะนี้อย่างไร (เช่น ระบบในต่างประเทศ หรือ Open Source Project)
- วิเคราะห์จุดแข็ง/จุดอ่อนของระบบเดิม หรือข้อจำกัดของแนวทางที่ใช้อยู่ในปัจจุบัน
- ระบุว่าระบบที่นักศึกษาจะพัฒนา มีการต่อยอดหรือแก้ไขปัญหจากระบบเดิมอย่างไร (Gap Analysis)

2. ศึกษาข้อมูลและ Business Use Case

- กำหนดปัญหาทางธุรกิจหรือคำถามที่ต้องการตอบ
- ระบุผู้ใช้งาน (เช่น ผู้บริหาร ฝ่ายการตลาด ฯลฯ)
- ระบุ Data Product ที่จะพัฒนา (Dashboard / API / ML)

3. วิเคราะห์แหล่งข้อมูล Data Source Analysis

- ระบุแหล่งข้อมูลอย่างน้อย 3 แหล่ง
- ระบุประเภทข้อมูล (File / API / Database / Streaming / Simulated)
- กำหนดรูปแบบการอัปเดตข้อมูล (Full / Incremental / Streaming)

4. ออกแบบสถาปัตยกรรม Architecture Design (End-to-End)

- ออกแบบ Medallion Architecture (Bronze–Silver–Gold)
- ออกแบบ Data Flow ตั้งแต่ Ingestion → Warehouse → Data Product
- วางแผนแนวทาง Incremental
- วางแนวคิด Orchestration (Task flow + schedule)

5. ออกแบบ Data Model / Data Warehouse

- กำหนด Grain คือ ระดับรายละเอียดของข้อมูลที่เก็บใน Fact Table
=> “1 แถวใน Fact Table แทนเหตุการณ์อะไร” เป็นการกำหนดว่าข้อมูลมีความละเอียดแค่ไหน
- ออกแบบ Fact & Dimension Tables และร่าง Star Schema

สิ่งที่ต้องส่ง

- Project Proposal (ตาม Template ที่กำหนด รายละเอียดอยู่ในภาคผนวก)
- **Submission Due Date: จันทร์ 16 มีนาคม 2569 เวลา 09:00 น.**

Phase 2: Data Pipeline & Warehouse Implementation (สัปดาห์ที่ 3 – 6)

เป้าหมาย ทำให้ระบบ Data Pipeline ทำงานได้จริง

1. พัฒนา Data Ingestion

- สร้างระบบที่รองรับการเปลี่ยนแปลงของข้อมูล โดยเน้นการทำ Incremental Load (โหลดเฉพาะข้อมูลใหม่) มีกลไกป้องกันข้อมูลซ้ำ (Idempotency) และจัดการข้อมูลแบบ Batch
Bonus: สร้างระบบที่รองรับข้อมูลแบบ Streaming หรือ Micro-batch

2. พัฒนา Data Transformation

- สร้างระบบเพื่อรวบรวมข้อมูลจากหลายแหล่ง จัดการทำความสะอาดข้อมูล และปรับโครงสร้างข้อมูล
- สร้างระบบตรวจสอบคุณภาพข้อมูล (Data Quality)

3. พัฒนา Data Warehouse

- พัฒนาล้างข้อมูลที่สามารถรองรับการโหลดข้อมูลใหม่เข้าไปได้อย่างถูกต้อง

4. เน้น Incremental Processing

- ใช้ Timestamp / Merge / Upsert / SCD

สิ่งที่ต้องแสดง

- Pipeline ทำงานครบจากแหล่งข้อมูลจนถึงคลังข้อมูล (Manual run ได้)
- ตัวอย่างข้อมูลในแต่ละ Layer
- ตัวอย่าง Query เชิงธุรกิจ

Phase 3: Data Product & Orchestration Implementation (สัปดาห์ที่ 7)

เป้าหมาย เชื่อมทุกขั้นตอนเป็น Workflow และสร้าง Data Product

1. พัฒนา Data Product อย่างน้อย 1 รูปแบบ

- สร้าง Dashboard / BI หรือ API ที่เชื่อมต่อโดยตรงกับคลังข้อมูลที่ได้พัฒนาใน Phase ก่อนหน้านี้ หรือ
- สร้าง Machine Learning Model จากคลังข้อมูลที่เป็น Features Store

2. End-to-End Testing

- ทดสอบการไหลของข้อมูลตั้งแต่ต้นจนจบ เพื่อตรวจสอบความถูกต้องของตัวเลขใน Dashboard
- ทดสอบ Incremental (รันซ้ำ)

3. Orchestration Implementation

- นำ Pipeline ที่เสร็จแล้วไปวางบนเครื่องมือควบคุม (Cron / Airflow / Prefect) เพื่อให้ทำงานตามเวลาที่กำหนด (Schedule)
- พร้อมทั้งมีระบบ Logging และ Error Handling (Retry / Fail handling)

สิ่งที่ต้องแสดง

- Screenshot หรือ Demo ของ Data Product
- ตัวอย่าง Log การทำงาน
- Workflow

Phase 4: Final Presentation (สัปดาห์ที่ 8)

เป้าหมาย ตรวจสอบความถูกต้อง สรุปผลงานและนำเสนอโครงการ

1. ตรวจสอบ Incremental และ Idempotency
2. ตรวจสอบ Data Quality
3. สรุปผลงานในรูปแบบรายงาน
4. เตรียมการนำเสนอ และจัดทำสื่อการนำเสนอ
5. จัดทำคลิป Demonstrate ผลงานทั้งหมด

Final Deliverables

- Source Code (Repository)
- รายงาน (ตาม Template ที่กำหนด รายละเอียดอยู่ในภาคผนวก)
- สื่อการนำเสนอ Presentation Slide
- คลิป Video Demonstration

- Video Demo ต้องแสดง
 - แหล่งข้อมูลต้นทาง
 - การรัน Pipeline (หรือผลการรัน) พร้อมแสดงวิธีการจัดการ Incremental Processing
 - ข้อมูลใน Bronze/Silver/Gold (บางส่วน)
 - Data Product (Dashboard / API / ML result)
- ข้อกำหนดตัวอย่าง
 - ความยาว: 5 - 8 นาที
 - รูปแบบ: Screen recording + voice (หรือ subtitle)
 - ส่งลิงก์ YouTube / Drive ผ่าน MyCourses
- Submission: พุธ 29 เมษายน 2569 เวลา 09:00 น.
- Presentation: พฤหัสบดี 30 เมษายน 2569 เวลา 14:00 - 16:00 น.

สรุปแผนการดำเนินโครงการ: Project Milestone & Expected Output

Phase	Weeks	Milestone	Expected Output
1. Planning	1 - 2	กำหนดโจทย์ธุรกิจ และออกแบบสถาปัตยกรรม	Project proposal ตาม Template โดยมีส่วนสำคัญ เช่น Architecture Diagram (Draft), Data Model (Draft) Submission: จันทร์ 16 มีนาคม 2569 เวลา 09:00 น.
2. Implementation	3 - 6	พัฒนา Pipeline และ Data Warehouse ให้ทำงานได้จริง	Raw Data Structure, Cleaned Tables, Data Quality Rules, Fact & Dimension Tables, Pipeline (Manual Run), Sample Business Queries
3. Integration	7	เชื่อมต่อ Workflow และสร้าง Data Product	Dashboard/API/ML Demo, Workflow Diagram, Log ตัวอย่าง
4. Finalization	8	ตรวจสอบความถูกต้องและนำเสนอโครงการ	Source Code, รายงานฉบับสมบูรณ์, Slide Presentation, คลิป Demonstration Submission: พุธ 29 เมษายน 2569 เวลา 09:00 น. Presentation: พฤหัสบดี 30 เมษายน 2569 เวลา 14:00 - 16:00

Project Topic Selection

นักศึกษาสามารถเลือกดำเนินโครงการได้ 2 รูปแบบ:

- เลือกจากหัวข้อตัวอย่างที่กำหนด: เช่น ระบบวิเคราะห์พลังงานอัจฉริยะ (MUICT), ระบบเฝ้าระวังสุขภาพ, หรือระบบตรวจจับการทุจริตทางการเงิน
- เลือกหัวข้ออิสระ: นักศึกษาสามารถเสนอหัวข้ออื่นที่สนใจได้เอง โดยต้องคงองค์ประกอบทางเทคนิคให้ครบถ้วนตามข้อกำหนด ดังนี้:
 - อยู่ในขอบเขต Data Engineering / Data Pipeline
 - มีอย่างน้อย 3 แหล่งข้อมูล ซึ่งมีระบบ Incremental
 - มี Data Pipeline + Data Warehouse + Data Product
 - ต้องได้รับการอนุมัติใน Phase 1

ตัวอย่างของโครงการด้าน Data Engineering

โครงการ	วัตถุประสงค์ (Objective)	ตัวอย่าง Data Sources	ตัวอย่าง Data Products
1. ระบบวิเคราะห์พลังงานในอาคารอัจฉริยะ	ลดการใช้พลังงานที่สูญเสียในอาคาร โดยเปรียบเทียบการใช้ไฟฟ้าจริงกับตารางการใช้งานห้องเรียน	เซนเซอร์ IoT ในอาคาร (Stream), ข้อมูลการจองห้อง (API), ตารางเรียน/ตารางสอน (Excel)	Dashboard แสดงการใช้พลังงาน, ระบบแจ้งเตือนเมื่อพบการเปิดไฟในห้องว่าง
2. ระบบติดตามและช่วยเหลือนิสิต	ระบุนิสิตที่มีความเสี่ยงทางการเรียนต่ำกว่าเกณฑ์ โดยวิเคราะห์จากการเข้าใช้งานระบบการเรียนออนไลน์	บันทึกการคลิกใน LMS (JSON/Logs), ผลการเรียนรู้ (SQL), บันทึกการเข้าใช้ห้องสมุด/อาคาร (IoT)	ระบบ Success Score รายบุคคล, รายชื่อนิสิตที่ควรได้รับคำปรึกษาเร่งด่วน
3. ระบบวิเคราะห์และป้องกันอาชญากรรม	ระบุพื้นที่เสี่ยงอาชญากรรม (Hot Spots) เพื่อช่วยให้เจ้าหน้าที่วางแผนลาดตระเวนได้อย่างมีประสิทธิภาพ	รายงานคดีจากตำรวจ (SQL), บันทึกแจ้งเหตุ 911 (JSON), ข้อมูลประชากร (CSV/Parquet)	แผนที่ความร้อนอาชญากรรม (Heat Map), ตารางแนะนำเส้นทางลาดตระเวนรายวัน
4. ระบบเพิ่มประสิทธิภาพโซ่อุปทาน (Supply Chain)	ลดปัญหาคอขวดในระบบขนส่งและจัดการสินค้าคงคลัง โดยติดตามสถานะสินค้าจากผู้ผลิตถึงคลังสินค้า	บันทึกจากคลังสินค้า (SQL), สถานะการขนส่ง (Webhooks/API), ฐานข้อมูลประวัติการส่งสินค้า (Parquet)	ระบบคำนวณวันถึงที่หมาย (ETA) แบบแม่นยำ, ระบบแจ้งเตือนสินค้าขาดสต็อก
5. แพลตฟอร์มวิเคราะห์ความรู้สึกบนโซเชียลมีเดีย	วิเคราะห์กระแสสังคมและความรู้สึกของสาธารณชนต่อแบรนด์หรือนโยบายต่างๆ แบบ Real-time	API จาก social media (Stream), ข้อมูลจาก Reddit (JSON), ข่าวจาก RSS Feeds (XML)	Dashboard แสดงกราฟความรู้สึก (Sentiment) และคำค้นหาที่กำลังเป็นกระแส (Trending Topics)
6. ระบบแจ้งเตือนความเสี่ยงสิ่งแวดล้อมและประกันภัย	วิเคราะห์คุณภาพอากาศร่วมกับข้อมูลสุขภาพ และแจ้งเตือนภัยน้ำท่วมแก่ลูกค้าประกันภัยตามตำแหน่ง GPS	เซนเซอร์ PM2.5 (Streaming), ข้อมูลผู้ป่วย (Batch), แผนที่เขตพื้นที่น้ำท่วม (Static)	ระบบแจ้งเตือนภัยผ่านมือถือ (Push Notification), แผนที่วิเคราะห์ความเสี่ยงรายพื้นที่
7. ระบบเฝ้าระวังสุขภาพตามพื้นที่	ตรวจสอบและคาดการณ์การระบาดของโรคติดต่อในระดับท้องถิ่น เพื่อวางแผนรับมือเชิงรุก	ข้อมูลโรคระบาดจาก WHO (API), บันทึกการรักษาของโรงพยาบาล (CDC), ข้อมูลคุณภาพอากาศ (Stream)	Dashboard แสดงพื้นที่เสี่ยงโรคระบาด, ระบบแจ้งเตือนสถานการณ์สุขภาพท้องถิ่น
8. ระบบจัดการเกษตรแม่นยำ	เพิ่มผลผลิตทางการเกษตรโดยวิเคราะห์ข้อมูลเซนเซอร์ในดินร่วมกับภาพถ่ายดาวเทียม	เซนเซอร์ความชื้นในดิน (IoT), ภาพถ่ายดาวเทียม NDVI (Raster), ข้อมูลสภาพอากาศ (API)	แผนที่การให้ปุ๋ยและน้ำรายจุด (Prescription Map), API สำหรับสั่งการโดรนเกษตร
9. ระบบจัดการจราจรเมืองอัจฉริยะ	แก้ไขปัญหาการติดและเพิ่มความตรงต่อเวลาของรถสาธารณะด้วยการปรับสัญญาณไฟจราจรแบบ Real-time	GPS รถสาธารณะ (GTFS), ข้อมูลอุบัติเหตุ (เช่น ThaiRSC), ข้อมูลความหนาแน่นจากแอปฯ เรียกรถ (Batch)	แอปฯ คาดการณ์เวลารถเมล์เข้าป้าย, ระบบปรับสัญญาณไฟจราจรอัตโนมัติ
10. ระบบตรวจจับการทุจริตทางการเงิน	ป้องกันความเสียหายทางการเงินโดยการตรวจสอบและระบุธุรกรรมที่ผิดปกติในเสี้ยววินาที	ข้อมูลการทำธุรกรรม (Stream), ข้อมูลพฤติกรรมลูกค้า (SCD Type 2), บัญชีดำ IP/ร้านค้า (API/CSV)	ระบบระบุธุรกรรมต้องสงสัยอัตโนมัติ, รายงานสรุปเคสที่เข้าข่ายการทุจริต

ข้อกำหนดด้านข้อมูล

แต่ละโครงการต้องมี

- อย่างน้อย 3 แหล่งข้อมูล
- อย่างน้อย 1 แหล่งข้อมูล ต้องเป็น “ข้อมูลแบบต่อเนื่อง (Batch Incremental)” คือ การรับและประมวลผลข้อมูลที่เพิ่มเข้ามาใหม่เป็นช่วงเวลา (เป็นรอบๆ) โดยระบบจะดึงหรือโหลด เฉพาะข้อมูลใหม่หรือข้อมูลที่มีการเปลี่ยนแปลง ในแต่ละรอบ (*Incremental Load*) แทนการโหลดข้อมูลทั้งหมดทุกครั้ง แนวทางนี้ช่วยลดเวลาในการประมวลผล ลดการใช้ทรัพยากร และสะท้อนรูปแบบการทำงานจริงของระบบข้อมูลในองค์กร เช่น
 - ไฟล์รายวัน / รายชั่วโมง
 - เช่น ไฟล์รายการขาย หรือรายการสินค้าคงคลัง ที่สมมติว่าระบบจะไปดึงข้อมูลจากระบบ POS ทุกๆ 1 ชั่วโมง หรือสัปดาห์
 - ข้อมูลอัปเดตเพิ่มแบบต่อเนื่อง
 - เช่น ระบบส่งไฟล์ข้อมูลลูกค้าใหม่ หรือรายการสั่งซื้อใหม่เข้ามาเป็นระยะ โดย Pipeline จะต้องโหลดเฉพาะข้อมูลที่เพิ่มเข้ามาใหม่ และหลีกเลี่ยงการโหลดข้อมูลซ้ำ (เช่น ใช้วันที่/เวลา หรือรหัสรายการเป็นตัวตรวจสอบ)
 - ข้อมูลจำลอง (Simulated Data)
 - เช่น จำลองข้อมูลเซนเซอร์จาก 10 จุดตรวจวัด โดยสรุปข้อมูลส่งเป็นรายชั่วโมง หรือสร้างสคริปต์ที่สร้างข้อมูลใหม่และบันทึกลงไฟล์เดอร์ทุกๆ ช่วงเวลา เพื่อให้ระบบทำงานเสมือนรับข้อมูลจริงอย่างต่อเนื่อง
- แหล่งข้อมูลสามารถเป็น
 - Open Dataset เช่น Kaggle, UCI Machine Learning Repository, data.gov, data.go.th, data.bangkok.go.th เป็นต้น
 - ไฟล์ข้อมูล (CSV/JSON/Parquet)
 - API
 - ข้อมูลจำลอง
- **คะแนนโบนัส (Bonus):** มีอย่างน้อย 1 แหล่งข้อมูล ที่เป็น “ข้อมูลสตรีมมิ่ง (Streaming Data)” คือ ข้อมูลที่ถูกสร้างและส่งเข้ามาอย่างต่อเนื่องแบบใกล้เคียงเวลาจริง (Near Real-time) โดยข้อมูลจะเข้ามาทีละรายการหรือเป็นชุดขนาดเล็กตามเหตุการณ์ที่เกิดขึ้น ระบบต้องสามารถรับและประมวลผลข้อมูลอย่างต่อเนื่อง หรือประมวลผลเป็นช่วงเวลาสั้นๆ (Micro-batch) แทนการรวบรวมข้อมูลเป็นรอบใหญ่แบบ Batch รูปแบบนี้เหมาะสำหรับงานที่ต้องติดตามสถานการณ์ปัจจุบันหรือสรุปผลแบบใกล้เคียงเวลาจริง เช่น
 - Time-based Streaming ข้อมูลถูกส่งเข้ามาเป็นช่วงเวลาสม่ำเสมอ เช่น ทุกๆ 1 นาที หรือ 5 นาที
 - เช่น ราคาหุ้นที่อัปเดตทุกๆ 5 นาที ข้อมูลสภาพอากาศหรือค่าฝุ่น PM2.5 ที่รายงานทุกๆ 10 นาที หรือ ข้อมูลการใช้พลังงานไฟฟ้าหรือข้อมูลจากอุปกรณ์ IoT ที่ส่งค่าทุกช่วงเวลา
 - Event-based Streaming ข้อมูลถูกส่งเข้ามาทันทีเมื่อมีเหตุการณ์เกิดขึ้น โดยความถี่อาจไม่สม่ำเสมอ
 - เช่น ข้อมูลโพสต์หรือข้อความใหม่จาก Social Media การทำธุรกรรมในระบบ e-commerce หรือการชำระเงิน หรือ Log การใช้งานระบบ เช่น การ login, click, หรือการเข้าชมเว็บไซต์
 - ข้อมูลจำลอง (Simulated Streaming Data) กรณีไม่สามารถใช้แหล่งข้อมูลจริง สามารถจำลองข้อมูลให้ไหลเข้าระบบอย่างต่อเนื่องได้

- เช่น สคริปต์ที่สร้างข้อมูลธุรกรรมหรือข้อมูลเซนเซอร์ใหม่ทุกๆ ไม่กี่วินาทีหรือทุกนาที โปรแกรมที่ส่งข้อความจำลองเข้าสู่ระบบ (เช่น Kafka/Message Queue) เป็นช่วงเวลา การเพิ่มไฟล์หรือบันทึกข้อมูลใหม่ลงในโพลเดอร์เป็นระยะ เพื่อให้ระบบประมวลผลแบบต่อเนื่อง แต่ต้องมีความถี่ที่สูงมากและใช้รูปแบบ Micro-batch โดยมี Flow การทำงาน ดังนี้
 - *Generate*: ใช้ Python Script (เช่น Mimesis/Faker) สร้าง Data จำลอง
 - *Produce*: ส่งข้อมูลเข้า Message Broker (เช่น Kafka, RabbitMQ หรือ Pub/Sub)
 - *Consume*: เขียนโปรแกรม (เช่น Spark Streaming หรือ Python Consumer) เพื่อดึงข้อมูลมาคำนวณทันที

ข้อกำหนดทางเทคนิค

- 1) โครงสร้างระบบข้อมูลแบบหลายชั้น
ต้องมีอย่างน้อย 3 ชั้น ตามแนวคิด Medallion Architecture

ระดับชั้น	คำอธิบาย	ผลลัพธ์ที่นักศึกษาต้องแสดง
Bronze (Raw Layer)	- เก็บข้อมูลต้นฉบับจากทุกแหล่ง - ไม่แก้ไขข้อมูล (Immutable) - เก็บตามรูปแบบเดิม (CSV/JSON/API dump) - จัดโครงสร้างโพลเดอร์ตามวัน/เวลา	- ไฟล์ต้นฉบับ - โครงสร้างโพลเดอร์ Raw
Silver (Clean/Transformation Layer)	- ทำความสะอาดข้อมูล เช่น ลบข้อมูลซ้ำ จัดการค่าว่าง ปรับ Data Type - จัดรูปแบบให้เป็นมาตรฐาน - ตรวจสอบความถูกต้องของข้อมูล - รวมข้อมูลจากหลายแหล่ง	- ตารางข้อมูลหรือไฟล์ข้อมูลที่ทำความสะอาดแล้ว (cleaned tables/files) - แสดงขั้นตอนการ Transform - Source Code หรือ รายงาน Data Quality อย่างน้อย 3 กฎ
Gold (Data Warehouse Layer)	- ออกแบบโครงสร้างข้อมูลเชิงธุรกิจ เช่น Star Schema เพื่อการวิเคราะห์ (Analytics-Ready) - อาจมีการ Aggregate ข้อมูล เพื่อรองรับการวิเคราะห์/รายงาน เชิง Business Logic - ถ้าวางรับ ML ระดับนี้อาจเป็น Feature Store	- ER Diagram / Start Schema (โดยควรมีอย่างน้อย 1 Fact Table และ 2 Dimension Tables) - ตัวอย่าง Query เชิงวิเคราะห์ หรือ Aggregated Views (เช่น ยอดขายรวมรายวัน, จำนวน User ที่ Active ต่อเดือน) - หรือ Feature Store (กรณี ML)

2) Incremental Processing (สำคัญ)

ระบบต้องสามารถ โหลดเฉพาะข้อมูลใหม่, ไม่เกิดข้อมูลซ้ำ, สามารถรันซ้ำได้อย่างปลอดภัย (Idempotent)
ตัวอย่างวิธี เช่น โหลดไฟล์ใหม่เท่านั้น, ใช้ Timestamp กรองข้อมูล หรือ Upsert / Merge

3) การตรวจสอบคุณภาพข้อมูล

อย่างน้อย 3 กฎการตรวจสอบ เช่น ตรวจสอบค่า Null, ตรวจสอบข้อมูลซ้ำ, ตรวจสอบช่วงค่าที่ถูกต้อง, ตรวจสอบโครงสร้างข้อมูล

4) การทำงานอัตโนมัติ (Orchestration)

Pipeline ต้องสามารถรันอัตโนมัติ โดยใช้เครื่องมือ เช่น Airflow, Prefect, Cron หรือเครื่องมือที่เหมาะสม และต้องมีการดูแลเรื่อง Logging และการจัดการข้อผิดพลาด

5) ส่วนเพิ่มเติม (คะแนนพิเศษ) สามารถเลือกทำเพิ่มเติมได้ เช่น:

- Streaming หรือ Micro-batch Processing
- Kafka หรือ Spark Streaming
- DWH advanced modelling such as SCD
- Deploy บน Cloud (AWS / GCP / Azure)
- Big Data Framework

แบบประเมินคะแนนโครงงาน (Scoring Rubric)

คะแนนรวม: 100 คะแนน

Phase	คะแนนเต็ม
Phase 1: Business Case, Data Source Analysis, Architecture & Star Schema Design	20
Phase 2: Ingestion, Incremental Load, Transformation, Data Quality & Warehouse	45
Phase 3: Data Product, Automation (Orchestration), Logging & E2E Testing	20
Phase 4: Documentation, Professional Presentation, System Demo Video	15
Total	100
+ คะแนนพิเศษ สูงสุด 10 คะแนน	

ระดับการประเมิน

- 4 = ดีมาก (ครบถ้วน ถูกต้อง เชื่อมโยงเชิงระบบ)
- 3 = ดี (ครบเกือบทั้งหมด มีรายละเอียดและใช้งานได้ มีข้อผิดพลาดเล็กน้อย)
- 2 = พอใช้ (ทำได้บางส่วน ยังไม่สมบูรณ์ ยังไม่ชัดเจน)
- 1 = ต้องปรับปรุง (ขาดส่วนสำคัญ หรือไม่ถูกต้อง)
- 0 = ไม่มี (ไม่เกี่ยวข้อง หรือใช้งานไม่ได้)

Phase 1: Business Understanding & Planning (20 คะแนน)						
ด้านการประเมิน	Weight	Level 4	Level 3	Level 2	Level 1	Level 0
Existing System / Benchmark	4	ศึกษา $\geq 2-3$ แหล่ง วิเคราะห์ ข้อดี-ข้อจำกัด และเชื่อมโยง ชัดเจนกับแนวทางออกแบบของ โครงงาน	ศึกษาอย่างน้อย 2 แหล่ง และอธิบายความเกี่ยวข้อง ได้ แต่การวิเคราะห์ยังไม่ลึก	กล่าวถึงระบบ/งานที่ เกี่ยวข้อง แต่เป็นการ สรุปทั่วไป ไม่มีการ เปรียบเทียบ	กล่าวถึงเล็กน้อย หรือไม่เกี่ยวข้อง ชัดเจน	ไม่มี การศึกษา
Business Understanding	4	Problem ชัดเจน ระบุ Stakeholders, Impact และ KPI ที่วัดผลได้ เชื่อมโยงกับ Data Solution อย่างเป็นระบบ	อธิบายปัญหาและ ผู้เกี่ยวข้องชัด มีเป้าหมาย วัดผลบางส่วน	ระบุปัญหาได้ แต่ยังไม่ ชัดเรื่อง Impact/KPI	ปัญหาเป็นเชิง เทคนิคมากกว่า ธุรกิจ หรือไม่ สอดคล้อง	ไม่มี Business Problem

Faculty of ICT, Mahidol University
ITDS344 Data Engineering and Infrastructures (Semester 2/2568)

ด้านการประเมิน	Weight	Level 4	Level 3	Level 2	Level 1	Level 0
Data Source Analysis	4	วิเคราะห์แหล่งข้อมูลครบ (โครงสร้าง, รูปแบบ, ความถี่, Data Quality, ความเสี่ยง PII) และระบุความท้าทายได้	อธิบายแหล่งข้อมูลครบ อย่างน้อย 3 แหล่ง และระบุ ประเภท/รูปแบบได้ชัด	ระบุแหล่งข้อมูลได้ แต่ไม่มีการวิเคราะห์เชิงลึก	ข้อมูลไม่ครบ หรือไม่ชัดเจน โครงสร้าง/รูปแบบ	ไม่มีการวิเคราะห์ข้อมูล
Architecture Design	4	มี Architecture Diagram ครบคลุม อย่างน้อย 3 ชั้น พร้อมอธิบายเหตุผลการออกแบบ	มี Diagram ครบ flow หลัก และอธิบายการทำงานได้	มี Diagram แต่ขาดบาง layer หรือ ความสัมพันธ์ไม่ชัด	Diagram ไม่สะท้อน data flow จริง หรือมีเพียงคำอธิบาย	ไม่มี Architecture
Data Warehouse Design	4	กำหนด Grain ชัดเจน ออกแบบ Fact/Dimension ถูกต้อง สอดคล้อง Business Question	มี Fact/Dimension ครบ และระบุ Grain ได้	มีโครงสร้างพื้นฐาน แต่ Grain หรือ ความสัมพันธ์ยังไม่ชัด	ออกแบบตารางไม่สอดคล้องกับโจทย์	ไม่มี Data Model

Phase 2: Data Pipeline & Warehouse Implementation (45 คะแนน)						
ด้านการประเมิน	Weight	Level 4	Level 3	Level 2	Level 1	Level 0
Data Ingestion	10	ดึงข้อมูลจากหลายแหล่งได้ครบ และทำงานได้จริง รองรับรูปแบบที่แตกต่าง (API/File/DB)	ดึงข้อมูลได้ครบอย่างน้อย 3 แหล่ง ทำงานได้ แต่การจัดการรูปแบบยังไม่ยืดหยุ่น (มีรูปแบบเหมือนกันหมด)	ดึงข้อมูลได้มากกว่า 1 แหล่ง แต่ยังพบความไม่ถูกต้องบ้าง	ดึงข้อมูลได้แค่แหล่งเดียว และข้อมูลไม่ถูกต้อง	ไม่มี ingestion
Incremental / Idempotency	10	มี Incremental Load ถูกต้อง มี Idempotency และป้องกันข้อมูลซ้ำ	มี Incremental โหลด เฉพาะข้อมูลใหม่ได้ถูกต้อง	มีการอัปเดตข้อมูล แต่ยังเป็น full load เป็นหลัก	โหลดข้อมูลเพิ่มใหม่ไม่ได้	ไม่มีการจัดการ update
Data Transformation	10	มีขั้นตอน Clean, Transform ชัดเจน และโครงสร้างข้อมูลเป็นมาตรฐาน (Bronze-Silver-Gold หรือเทียบเท่า)	มีการทำความสะอาดข้อมูล และมีการแปลงข้อมูลพื้นฐานเป็นส่วนใหญ่	มีการทำความสะอาดข้อมูลพื้นฐานได้ถูกต้อง แต่มีเพียงเล็กน้อย	มีการทำความสะอาดข้อมูลบ้าง แต่ข้อมูลยังคงไม่ถูกต้อง	ไม่มี transformation
Data Quality	5	มีการตรวจสอบคุณภาพข้อมูลพื้นฐาน มากกว่า 3 กฎ พร้อมการแสดงผล Log และจัดการ error ได้	มีการตรวจสอบคุณภาพข้อมูลพื้นฐาน ครบ 3 กฎ และแสดงผลชัดเจน	มีการตรวจสอบคุณภาพข้อมูลพื้นฐาน น้อยกว่า 3 กฎ	มีการตรวจสอบ แต่ ไม่มีระบบตรวจสอบชัดเจน	ไม่ตรวจสอบข้อมูล
Data Warehouse (Gold)	10	Warehouse ทำงานได้จริง Query ตอบโจทย์ธุรกิจได้ ถูกต้องตาม design และรองรับการรันซ้ำ	Warehouse ใช้งานได้ ถูกต้องตาม Schema ที่ได้ ออกแบบไว้ และสามารถ Query ใช้งานได้	โหลดข้อมูลได้แต่โครงสร้างหรือ ความสัมพันธ์ยังไม่สมบูรณ์ หรือข้อมูลไม่ถูกต้อง	Warehouse ไม่สอดคล้องกับ design	ไม่มี Warehouse

Phase 3: Data Product & Orchestration (20 คะแนน)

ด้านการประเมิน	Weight	Level 4	Level 3	Level 2	Level 1	Level 0
Data Product (i.e., Dashboard, API, ML)	10	เชื่อมต่อกับ Gold Layer โดยตรงและถูกต้อง ตอบโจทย์ Business Question ชัดเจน มี Insight ที่ช่วยในการตัดสินใจ และ UI/UX ใช้งานง่าย	สามารถใช้งานได้ตามเป้าหมายธุรกิจที่วางไว้ ข้อมูลมีความถูกต้องและเชื่อมต่อกับ Data Warehouse ได้	แสดงผลข้อมูลทั่วไปได้ แต่ Insight ยังไม่ชัดเจน หรือยังไม่สามารถตอบ โจทย์ธุรกิจได้ครบถ้วน	การเชื่อมต่อไม่สมบูรณ์ ข้อมูลที่แสดงผลมีความคลาดเคลื่อน หรือ UI ใช้งานยากมาก	ไม่มี Data Product
Orchestration (Automation)	5	ใช้เครื่องมือควบคุม (Airflow/Cron/ฯลฯ) จัดการ Task Dependencies ได้ สมบูรณ์ ตั้ง Schedule ทำงาน อัตโนมัติ และรองรับ Retry Logic	มีระบบ Scheduler ที่ทำงานได้จริงตามเวลาที่กำหนด และสามารถควบคุม ลำดับการรันของ Pipeline ได้	ระบบรันแบบ Manual เป็นหลัก หรือการตั้ง Schedule ยังมีข้อผิดพลาด (เช่น รัน ช้าซ้อน)	มีเพียงแนวคิดหรือสคริปต์พื้นฐาน แต่ไม่สามารถทำให้ทำงานอัตโนมัติได้จริง	ไม่มีระบบ Automation
Testing / Logging / Monitoring	5	มีระบบ Logging ครบทุกขั้นตอน (Success/Fail), มีการแจ้งเตือน (Alerting) เมื่อ Pipeline พัง และมีเอกสารผลการทดสอบ End-to-End Test	มีการทำ Logging บันทึก สถานะการทำงาน และมีการตรวจสอบความถูกต้องของข้อมูล (Data Reconciliation) หลังรันเสร็จ	มีการตรวจสอบแบบ Manual หรือมี Log เพียงบางส่วน ทำให้ยากต่อการ Debug เมื่อระบบมีปัญหา	มีการตรวจสอบเล็กน้อย ไม่เป็นระบบ และไม่มี หลักฐานการทดสอบที่ชัดเจน	ไม่มีระบบ Monitoring

Phase 4: Final Presentation & Documentation (15 คะแนน)						
ด้านการประเมิน	Weight	Level 4	Level 3	Level 2	Level 1	Level 0
Documentation	5	เอกสารละเอียดครบถ้วน มี Architecture Diagram ที่ถูกต้อง, อธิบาย Flow ข้อมูล, วิธีทำ Incremental และกฎ DQ ชัดเจน	ครบตามหัวใจสำคัญที่กำหนด แผนภาพถูกต้องแต่อาจขาดรายละเอียดปลีกย่อย	มีเอกสารและแผนภาพ แต่เนื้อหาน้อย หรือ อธิบายไม่ชัดเจนในบางส่วน	เอกสารไม่ครบถ้วน ตามหัวข้อที่บังคับ หรือแผนภาพไม่ตรงกับงานจริง	ไม่มี
Presentation	5	นำเสนออย่างเป็นมืออาชีพ สรุปประเด็นทางเทคนิคและธุรกิจได้ดี ตอบคำถามได้แม่นยำและตรงประเด็น	อธิบายเนื้อหาได้ครบถ้วนตามลำดับ เข้าใจง่าย ตอบคำถามได้ส่วนใหญ่	นำเสนอติดขัดบ้าง เนื้อหาเยอะเกินไปหรือสรุปไม่ตรงประเด็น ตอบคำถามไม่ชัดเจน	อ่านตามสไลด์ ขาดการเตรียมตัว หรือไม่สามารถตอบคำถามพื้นฐานได้	ไม่นำเสนอ
System Demo	5	คลิปคุณภาพดี แสดงให้เห็น Flow ตั้งแต่ต้นจนจบ (E2E) รวมถึงโซลูการทำงานของ Automation และผลลัพธ์ที่ Data Product	แสดงการทำงานของระบบได้ครบถ้วน แต่อาจจะข้ามรายละเอียดเชิงเทคนิคบางอย่างไป	แสดงเฉพาะบางส่วน ของระบบ หรือคลิปไม่ลื่นไหล เห็นภาพไม่ชัดเจน	คลิปสั้นเกินไป หรือไม่สามารถพิสูจน์ได้ว่าระบบทำงานอัตโนมัติจริง	ไม่ส่งวิดีโอ

ภาคผนวก

รายงานข้อเสนอโครงการ Proposal Report (Phase 1)

- หน้าปกรายงาน
- สารบัญ
- Chapter 1: Introduction
 - 1) ความสำคัญของปัญหา (Motivation)
 - 2) Business Problem
 - 3) วัตถุประสงค์ (Objective)
 - 4) ผลลัพธ์ที่คาดหวัง (Expected Output) เช่น Dashboard/API/LML, etc
 - 5) ประโยชน์ที่คาดหวังของระบบ (Expected Benefits)
- Chapter 2: Background
 - 1) การศึกษาโครงการที่เกี่ยวข้อง (Existing Projects/Benchmark) อย่างน้อย 3 โครงการ เพื่อวิเคราะห์หา Gap Analysis ข้อจำกัดหรือช่องว่างของระบบที่มีอยู่
- Chapter 3: Design
 - 1) Data Source Analysis
 - (นักศึกษาสามารถออกแบบการนำเสนอข้อมูลนี้เองได้ ยกตัวอย่างเช่น ทำเป็นตารางสรุปแหล่งข้อมูลแต่ละรายการ)

Data (Name)	Source	Access Method	Update Frequency	Format	Limitation
1. AQI Index	AQI App	API access	Updated once a day	Semi/Unstructure	e.g., free to get but can get only 2000 data points
2.					
3.					

2) Data Structures:

- โครงสร้างและคำอธิบายฟิลด์ข้อมูล (Data Dictionary) ยกตัวอย่าง Filed สำคัญ และระบุ Primary Key ให้ชัดเจน

3) Architecture Design:

- แผนภาพสถาปัตยกรรมแบบ End-to-End (Medallion Architecture)

- Incremental Concept (แนวคิด)
- Orchestration Concept (แนวคิด)

4) Data Warehouse Design:

- การออกแบบ Star Schema (Fact & Dimension Tables)
- ระบุ Grain ของข้อมูล

5) Challenges & Risk

- อธิบายว่าปัญหาและอุปสรรคที่คาดว่าจะต้องเจอในระหว่างการทำโครงการ

● Chapter 4: Implementation Plan

- 1) แผนการดำเนินงาน ที่มีการระบุ Timeline และผู้รับผิดชอบที่ชัดเจน
- 2) ระบุเทคโนโลยีที่เลือกใช้

● Reference: ระบุ reference ที่เกี่ยวข้อง

รายงานฉบับสมบูรณ์ Final Report (Phase 4)

- หน้าปกรายงาน
- สารบัญ
- Chapter 1-3:
 - 1) ปรับปรุงเนื้อหาจาก Proposal ให้เป็นปัจจุบันตามที่พัฒนาจริง
- Chapter 4: Data Pipeline Implementation
 - 1) Data Ingestion: รายละเอียดการดึงข้อมูลแต่ละแหล่ง
 - 2) Incremental Processing: วิธีการจัดการข้อมูลใหม่และกลไก Idempotency
 - 3) Data Transformation: ขั้นตอนการทำความสะอาดและแปลงข้อมูล
 - 4) Data Quality Report: กฎการตรวจสอบคุณภาพข้อมูลทั้ง 3 กฎและผลการทดสอบ (Screenshots/Logs)
 - 5) Data Warehouse: รายละเอียดของ Start Schema และตัวอย่างของ Business Queries ที่สร้างจาก Data Warehouse (Gold Layer) พร้อมผลลัพธ์ของ Queries
- Chapter 5: Automation & Data Product
 - 1) Orchestration: รายละเอียดการตั้งค่า Scheduler (เช่น Airflow DAGs), Workflow Diagram, และระบบ Logging/Monitoring พร้อมตัวอย่างของการจัดการ Errors (ถ้ามี)

2) **Data Product:** รายละเอียด Dashboard, API หรือ Model ที่พัฒนาขึ้น แสดง Use Case และตัวอย่างของ Output พร้อมผลการวิเคราะห์ข้อมูลที่ได้จาก Data Product นั้นๆ

- **Chapter 6: Conclusion & Challenges:**

- 1) สรุปผลการดำเนินงาน (อ้างอิงตามแผนการดำเนินงาน)
- 2) ระบุ ปัญหาที่พบ และแนวทางการแก้ไข
- 3) สิ่งที่ได้เรียนรู้จากการทำโครงการ (ขอให้ระบุแยกของนักศึกษาแต่ละคน)

- **Reference:** ระบุ reference ที่เกี่ยวข้อง

- **Appendix** (ถ้ามี)

FAQ คำถามที่พบบ่อยเกี่ยวกับโครงการ Data Engineering

ด้านความเข้าใจทางธุรกิจ

Q: ต้องมี Problem Statement ไหม?

A: ต้องมี และต้องชัดว่า ปัญหาคืออะไร ใครได้รับผลกระทบ ทำไม Data ถึงช่วยแก้ไขได้
ถ้าไม่มี problem = โปรเจกต์จะกลายเป็นแค่ “ทำระบบเฉย ๆ”

ด้านข้อมูล (Data Sources & Ingestion)

Q: จำเป็นต้องใช้ข้อมูลจริงจาก API ทั้งหมดเลยไหม?

A: ไม่จำเป็น สามารถผสมกันได้ เช่น แหล่งที่ 1 เป็น API, แหล่งที่ 2 เป็นไฟล์ CSV จาก Kaggle และแหล่งที่ 3 เป็น Simulated Data (ข้อมูลจำลอง) แต่ต้องมีอย่างน้อย 1 แหล่งที่เป็นแบบ Incremental (ข้อมูลใหม่เพิ่มขึ้นเรื่อยๆ) นักศึกษาต้องสามารถอธิบายที่มาของข้อมูล โครงสร้างต้องมีความชัดเจน และสอดคล้องกับปัญหาที่ต้องการแก้ไข

Q: ถ้าไม่มีข้อมูลจริงทำอย่างไร?

A: สามารถใช้ Open Data หรือสร้าง Mock Data ด้วย AI หรือเครื่องมืออื่นได้

Q: ข้อมูลจำลอง (Simulated Data) ต้องทำอย่างไรถึงจะนับเป็น Incremental?

A: ตัวอย่าง นักศึกษาเขียน Python Script ที่สร้างไฟล์ใหม่ (เช่น sales_20260223_1000.csv) ทิ้งไว้ในโฟลเดอร์ตามช่วงเวลา แล้วให้ Pipeline ตรวจจับและดึงเฉพาะไฟล์ใหม่นี้เข้าไป นักศึกษาต้องสามารถอธิบาย Logic ของการ Generate ข้อมูลที่นักศึกษาจำลองไว้ได้ และข้อมูลควรมีขนาดและความซับซ้อนที่เหมาะสม

Q: ต้องทำ Streaming ทุกทีมไหม?

A: ไม่จำเป็น เป็นเพียง Bonus

Q: ต้องออกแบบระบบให้รองรับการรับข้อมูลแบบ Incremental จำเป็นไหม?

A: จำเป็น (เป็นส่วนหลักของคะแนน)

Q: ต้องมีข้อมูลขนาดใหญ่ไหม?

A: ไม่จำเป็น เน้นแนวคิด Pipeline และ Architecture มากกว่าปริมาณข้อมูล

ด้านสถาปัตยกรรม (Architecture)

Q: ถ้าทำทุกอย่างในตารางเดียวเลยได้ไหม?

A: ไม่ได้ โครงการนี้เน้นการจัดการข้อมูลแบบอย่างน้อย 3 ชั้น เพื่อฝึกทักษะการแยกหน้าที่ของข้อมูล (Separation of Concerns) หากทำชั้นเดียวจะโดนหักคะแนนตามเกณฑ์ ไม่ได้บังคับว่าต้องทำแบบ Medallion Architecture แต่อย่างน้อยๆ ก็ควรมีระดับ Raw, Clean และ Serving (Data Warehouse) Layer

Q: Star Schema จำเป็นต้องมีไหม และต้องเป๊ะตามทฤษฎีไหม?

A: ถ้าเป็น Analytics Project ต้องมี แต่ถ้าเป็น ML อย่างเดียว อาจใช้ Feature Table แทนได้ โดย Star Schema ต้องมี 1 Fact Table (เก็บตัวเลขการวัด) และ 2 Dimension Tables (เก็บข้อมูลบริบท) เป็นอย่างน้อย โดยต้องนิยาม Grain (รายละเอียด 1 แถวคืออะไร) ให้ชัดเจนด้วย

Q: Idempotency คืออะไรกันแน่?

A: คือคุณสมบัติที่ว่า "ไม่ว่าคุณจะรัน Pipeline เดิมซ้ำกี่ครั้ง ผลลัพธ์ในฐานะข้อมูลต้องเหมือนเดิม" เช่น ถ้ารันซ้ำแล้วข้อมูลยอดขายต้องไม่เบิ้ลเป็นสองเท่า ระบบต้องฉลาดพอที่จะเช็กว่าข้อมูลนี้เคยถูกโหลดไปหรือยัง

Q: ทำไมต้องทำ Incremental Load?

A: ในโลกจริง ข้อมูลมีขนาดมหาศาล (Big Data) เราไม่สามารถโหลดข้อมูลทั้งหมดใหม่ทุกวันได้ การโหลดเฉพาะส่วนที่เพิ่มมาใหม่จะช่วยประหยัดทรัพยากรและเวลา

Q: ต้องทำ Data Quality แค่ไหน?

A: อย่างน้อยต้องมีการตรวจสอบ เช่น Missing, Duplicate หรือ Validation พื้นฐาน

Q: ในทีมมีแค่ 2 คน ต้องทำ Pipeline ซับซ้อนแค่ไหน และต้องดึงข้อมูลจากกี่แหล่งข้อมูล?

A: ไม่จำเป็นต้องซับซ้อน เน้นให้ระบบทำงานครบ ถูกต้อง และอธิบายได้ โดยยังคงต้องดึงข้อมูลจากอย่างน้อย 3 แหล่งข้อมูล

ด้านเครื่องมือและการทำงานอัตโนมัติ (Orchestration)

Q: ต้องใช้เครื่องมืออะไร? (Airflow, Spark, Kafka ฯลฯ)

A: เลือกใช้ได้ตามความเหมาะสม เช่น Python, SQL, dbt, Pandas, Power BI, etc. ไม่จำเป็นต้องใช้เครื่องมือระดับ Big Data แต่ต้องมีการแสดงให้เห็นว่ามีระบบ Logging (บันทึกว่ารันสำเร็จไหม) และ Error Handling (ถ้าพังต้องทำอะไร)

Q: ถ้ารัน Pipeline ซ้ำแล้วข้อมูลซ้ำ จะมีผลไหม?

A: มีผล จะถูกหักคะแนนตามเกณฑ์ เพราะระบบต้องรองรับ Incremental และต้องไม่เกิดข้อมูลซ้ำ

ด้าน Data Product

Q: ต้องทำ Machine Learning ที่มีความแม่นยำถูกต้องมากๆ ใช้หรือไม่?

A: ไม่ใช่จุดประสงค์หลัก โครงการนี้เน้นด้าน DE หากนิสิตทำ ML สิ่งที่จะตรวจคือ "ข้อมูลที่ป้อนเข้า Model สะอาดไหม" และ "Pipeline อัปเดต Feature ใหม่ๆ ให้ Model อัตโนมัติหรือเปล่า" (Feature Store) มากกว่าความแม่นยำของ Model

Q: Dashboard ถือเป็น Data Product ไหม?

A: ถือว่าใช่ ถ้า Dashboard ดังกล่าว ตอบโจทย์ธุรกิจ ใช้งานได้จริง ไม่ใช่แค่กราฟโชว์ข้อมูลดิบ